

Strukturni dizajn paterni

1. Adapter

Adapter je obrazac dizajna koji omogućava saradnju između različitih interfejsa. Konkretno u našem sistemu, sistem direktno poziva email servis ili printer bez posrednika. Ukoliko se promijeni email provider (npr. sa Gmail SMTP na SendGrid), mora se mijenjati kod svuda u sistemu.

Moguće rješenje je kreiranje IEmailServis i IStampacServis interfejsa. Za svaki vanjski servis napravi se Adapter koji "prevodi" taj interfejs. Ostatak sistema koristi samo interfejs i ne zna koji konkretni servis je ispod.

2. Bridge

Bridge odvajala apstrakciju od njene implementacije kako bi se obje mogle mijenjati nezavisno. Umjesto jedne hijerarhije klasa koja raste eksponencijalno, Bridge kreira dvije odvojene hijerarhije — jednu za tipove notifikacija i jednu za komunikacijske kanale.

Moguće rješenje: dvije nove hijerarhije: Notifikacija (apstrakcija — šta se šalje: potvrda termina, otkazivanje, podsjetnik) i IKanalSlanja (implementacija — kako se šalje: email, SMS, push notifikacija). Svaka kombinacija radi bez kreiranja posebnih klasa poput TerminEmailNotifikacija, TerminSMSNotifikacija itd.

3. Composite

Trenutna klasa MedicinskaUsluga modelira samo jednu medicinsku uslugu, dok u stvarnom medicinskom okruženju, ovo nije dovoljno. Postoje složene usluge koje se sastoje od više jednostavnijih - na primjer, "Sistematski pregled" obuhvata fizički pregled, krvnu sliku, EKG i RTG snimak, a svaka od tih stavki je zasebna usluga sa svojom cijenom, opisom i ocjenom. Trenutni model ne može prikazati tu hijerarhiju, što znači da se složene usluge moraju ručno razbijati.

Composite patern omogućava da MedicinskaUsluga može biti jednostavna ili složena usluga koja sadrži kolekciju drugih usluga. Uvodi se zajednički interfejs IUsluga koji implementiraju i JednostavnaUsluga i SlozenaUsluga. Ključna prednost je da klijentski kod tretira obje vrste potpuno identično - može pozvati getDetaljiUsluge() na bilo kojoj, bez razlike radi li se o jednoj usluzi ili o paketu od deset usluga.

4. Decorator

Decorator dinamički dodaje odgovornosti objektu bez izmjene njegove klase. Dekorator implementira isti interfejs kao originalna klasa, omotava (wrapa) originalni objekt i dodaje novu logiku prije ili nakon delegiranog poziva.

Moguća primjena u sistemu — formatiranje medicinskog nalaza:

Klasa Nalaz ima bazicnu metodu `stampajNalaz()` s fiksnim formatom. Uvode se dekoratori koji dinamički dodaju elemente: digitalni potpis doktora, PDF vođeni zig s logom poliklinike, QR kod za verifikaciju autentičnosti i oznaku za hitne slučajeve. Isti nalaz može dobiti bilo koju kombinaciju ovih dekoratora.

5. Facade

Facade patern pruža jednostavan interfejs prema složenom sistemu. U trenutnom dizajnu sistema, kada pacijent želi zakazati termin, on sam mora inicirati provjeru dostupnosti termina, kreiranje novog termina, provjeru liste čekanja, te na kraju slanje notifikacije. To znači da klasa Korisnik ima direktnu vezu i poznavanje internih detalja klasa Termin, ListaCekanja, MedicinskaUsluga i Notifikacija. Ovakav pristup narušava princip jedinstvene odgovornosti - klasa Korisnik zna previše o načinu rada ostatka sistema, što otežava održavanje i svaka promjena u bilo kojoj od tih klasa može zahtijevati izmjene i u klasi Korisnik.

Korištenjem Facade paterna kreira se nova klasa (npr. `ZakazivanjeTerminaFacade`) koja preuzima svu logiku, te samim tim klasa Korisnik više ne zna ništa o internim klasama. Ona samo pozove jednu metodu na Facade-u, npr. `zakaziTermin(pacijentID, doktorID, datum, uslugaID)`, i Facade interno obavlja sve korake. Time se drastično smanjuje zavisnost između klasa, kod postaje pregledniji i lakši za održavanje, a eventualne izmjene u procesu zakazivanja se rade na jednom mjestu.

6. Proxy

Primjena Proxy obrasca omogućava zaštitu osjetljivih podataka kao što su medicinski nalazi. Metode poput `getNalaz()` i `pregledajHistorijuPacijenta()` dostupne su bez ikakve provjere ko ih poziva i ima li pravo na pristup. Ne postoji mehanizam koji bi spriječio, recimo, jednog pacijenta da pristupi nalazima drugog pacijenta, ili medicinsku sestru da vidi nalaze koji su izvan njene nadležnosti.

U implementaciji se koristi interfejs `INalaz` koji definiše osnovne operacije nad nalazima, dok klase `Nalaz` i `NalazProxy` implementiraju taj interfejs. Na taj način klijent komunicira preko zajedničkog interfejsa, bez potrebe da zna da li koristi stvarni objekat ili Proxy. Proxy patern uvodi klasu `NalazProxy` koja stoji između klijenta i stvarne klase `Nalaz`. Svaki put kada neko pokuša pristupiti naložu, zahtjev prvo prolazi kroz Proxy koji provjerava ulogu i identitet korisnika. Doktor može vidjeti samo nalaze svojih pacijenata, pacijent samo svoje vlastite nalaze, a administrator ima potpuni pristup. Tek nakon uspješne provjere, Proxy proslijedi zahtjev stvarnoj klasi. Na taj način originalna klasa `Nalaz` ostaje nepromijenjena, a sigurnosna logika je centralizovana i jasno odvojena od poslovne logike.